

Получите новый уровень в карьере

Повышайте свою квалификацию

Хабр



Чему научиться в этом году?



512K+

Охват за 30 дней

OTUS

Развиваем технологии, обучая их создателей

704,71 Рейтинг

186 313 Подписчики

Подписаться



sproshchaev

22 часа назад

Контрактное тестирование на Kotlin: гайд для автоматизатора

🕒 7 мин 👁 5.1K

Блог компании OTUS, Kotlin*, Тестирование IT-систем*, Микросервисы*

Тutorial

Всем привет, меня зовут Сергей Прощаев, и в этой статье я расскажу про контрактное [тестирование на Kotlin](#) — подход, который экономит десятки часов отладки и спасает от неожиданных падений продакшена.

Я Tech Lead и руководитель направления Java | Kotlin разработки в FinTech, много лет проектирую и сопровождаю микросервисы в высоконагруженных системах. Преподаю на курсах разработки и архитектуры в OTUS, и регулярно вижу, как даже опытные команды спотыкаются на интеграционном тестировании. Вроде бы всё покрыто автотестами, `CI` зелёный, а после деплоя пользователи сыпят баг-репортами: «Приходит пустой список заказов», «Поле `customerId` вдруг стало `null`». Знакомая картина?

Сегодня разберём, как контрактные тесты решают эту проблему. Создадим потребительский контракт на Kotlin с помощью `Pact`, проверим, что бэкэнд ему соответствует, и обсудим типичные ошибки, из-за которых тесты превращаются в фикцию. А в конце поделюсь, где можно глубже прокачать навыки автоматизации тестирования на Kotlin, чтобы не наступать на эти грабли в своих проектах.

Почему интеграционные тесты не спасают

Интеграционные тесты проверяют, как сервисы общаются **здесь и сейчас**. Они хороши, когда у вас есть полная копия продакшен-окружения. Но стоит одному из сервисов изменить API — и ваши тесты всё ещё зелёные, потому что запускаются на старой версии. А в продакшене получаем сюрприз :((

Контрактное тестирование переворачивает логику. Вместо того чтобы проверять «работает ли интеграция в текущий момент», мы фиксируем **ожидания потребителя** от провайдера. Провайдер гарантирует, что не сломает эти ожидания в будущем. При этом каждая сторона тестирует только себя, что даёт быструю обратную связь прямо на этапе сборки.

Если говорить совсем просто:

- **Интеграционный тест** спрашивает: «Могут ли сервис А и сервис В договориться прямо сейчас?»
- **Контрактный тест** спрашивает: «Выполняет ли сервис В обещания, данные сервису А?»

Pact на Kotlin: с чего начать?

Среди инструментов для контрактного тестирования в JVM-мире де-факто стандартом стал **Pact**. Он позволяет описать контракт в виде JSON-файла, который генерируется на стороне потребителя, а затем верифицируется у провайдера. Для Kotlin есть удобная обёртка `pact-jvm-consumer-kotlin` с **DSL**, который выглядит почти как родной код.

Давайте сразу к практике. Представьте: у нас есть мобильное приложение (потребитель) и бэкэнд-сервис `user-service` (провайдер). Приложение ожидает, что эндпоинт `GET /users/{id}` вернёт JSON определённой структуры.

Шаг 1. Пишем контракт на стороне потребителя

Создаём тест, который описывает ожидания от API:

```
import au.com.dius.pact.consumer.dsl.PactDslJsonBody
import au.com.dius.pact.consumer.dsl.PactDslWithProvider
import au.com.dius.pact.consumer.junit5.PactConsumerTestExt
import au.com.dius.pact.consumer.junit5.PactTestFor
import au.com.dius.pact.core.model.RequestResponsePact
import au.com.dius.pact.core.model.annotations.Pact
import org.junit.jupiter.api.Test
import org.junit.jupiter.api.extension.ExtendWith
import org.springframework.web.client.RestTemplate

@ExtendWith(PactConsumerTestExt::class)
@PactTestFor(providerName = "user-service", port = "8080")
class UserServiceConsumerPactTest {

    @Pact(consumer = "mobile-app")
    fun getUserById(builder: PactDslWithProvider): RequestResponsePact {
        return builder
            .given("user with id 123 exists")
            .uponReceiving("a request for user 123")
            .path("/users/123")
            .method("GET")
            .willRespondWith()
            .status(200)
            .body(
                PactDslJsonBody()
                    .integerType("id", 123)
                    .stringType("name", "Alex")
                    .stringType("email", "alex@example.com")
                    .booleanType("active", true)
            )
            .toPact()
    }

    @Test
    @PactTestFor(pactMethod = "getUserById")
```

```

fun `should fetch user correctly`(mockServer: MockServer) {
    val response = RestTemplate().getForEntity("${mockServer.getUrl()}/users/123",
        assert(response.statusCode.is2xxSuccessful)
        // Здесь можно проверить маппинг в DTO
    }
}

```

Объяснить с  SourceCraft

Обратите внимание: мы не поднимаем реальный `user-service`. Мы поднимаем **мок-сервер**, который Pact генерирует из нашего контракта!

Тест проверяет, что наш клиентский код корректно обрабатывает ожидаемый ответ. После прогона теста в выходной директории (например, `target/pacts` для Maven или `build/pacts` для Gradle) появляется JSON-файл контракта.

Шаг 2. Верификация контракта на стороне провайдера

Теперь мы должны убедиться, что бэкэнд действительно отдаёт то, что обещано. Для этого на стороне провайдера пишем тест верификации:

```

import au.com.dius.pact.provider.junit5.HttpTestTarget
import au.com.dius.pact.provider.junit5.PactVerificationContext
import au.com.dius.pact.provider.junit5.PactVerificationInvocationContextProvider
import au.com.dius.pact.provider.junitsupport.Provider
import au.com.dius.pact.provider.junitsupport.State
import au.com.dius.pact.provider.junitsupport.loader.PactBroker
import org.junit.jupiter.api.BeforeEach
import org.junit.jupiter.api.TestTemplate
import org.junit.jupiter.api.extension.ExtendWith
import org.springframework.boot.test.context.SpringBootTest
import org.springframework.boot.test.web.server.LocalServerPort

@Provider("user-service")
@PactBroker(host = "localhost", port = "9292") // если используется Pact Broker
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class UserServiceProviderPactTest {

    @LocalServerPort
    var port: Int = 0

    @BeforeEach
    fun setup(context: PactVerificationContext) {
        context.target = HttpTestTarget("localhost", port)
    }

    @TestTemplate
    @ExtendWith(PactVerificationInvocationContextProvider::class)
    fun verifyPacts(context: PactVerificationContext) {
        context.verifyInteraction()
    }

    @State("user with id 123 exists")
    fun `prepare user 123`() {
        // здесь подготавливаем данные в БД, например через Testcontainers
    }
}

```

```
}  
}
```

[Объяснить с 🧑🏻‍🔬](#)

Этот тест поднимает реальный Spring Boot контекст (или его часть) и прогоняет запросы, описанные в контракте. Если реальный ответ отличается от ожидаемого, тест падает с чётким diff-ом.

Вот как выглядит процесс в целом (рис. 1):

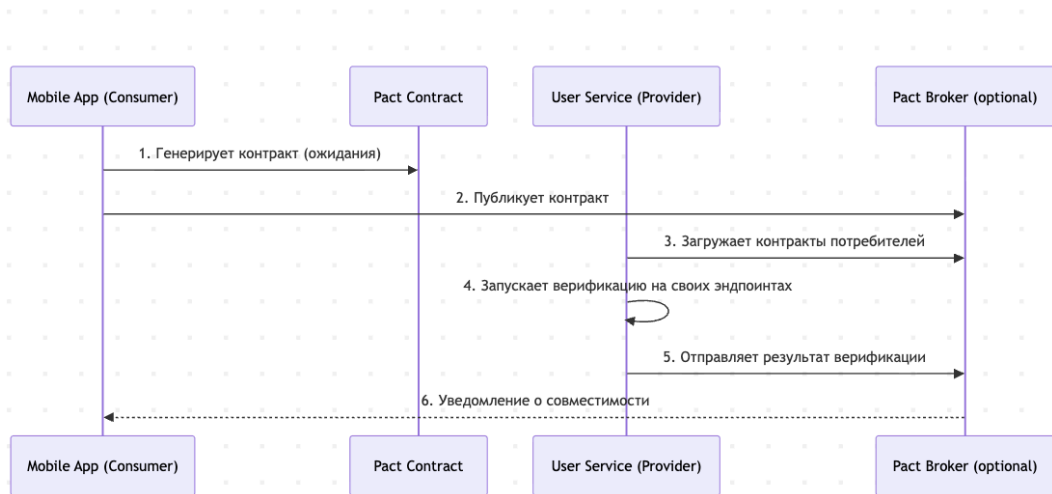


Рис. 1 Процесс контрактного тестирования с Pact

Где чаще всего ошибаются и как этого избежать?

За время работы с Pact я выделил для себя основные ошибки. Вот главные.

1. Слишком жёсткий контракт.

Новички часто проверяют каждое поле, включая служебные (например, `timestamp` с точностью до миллисекунд). При каждом деплое провайдера контракт ломается, команда устаёт чинить тесты и отключает их. Правильно — проверять только то, что реально нужно потребителю.

Используйте `eachLike`, `minArrayLike` и другие гибкие матчеры.

2. Забывают про состояния провайдера.

Контракт начинается с `given("user with id 123 exists")`. Если на стороне провайдера не реализован метод с аннотацией `@State`, верификация упадёт с загадочной ошибкой 404.

Обязательно готовьте данные в соответствующих методах.

3. Тестируют только `happy path`.

Потребитель должен описать не только успешный ответ, но и сценарии ошибок: 404, 400, 500. Без этого провайдер может втихую изменить формат ошибки, и клиентское приложение не сможет его распарсить.

4. Публикуют контракты в общую папку без версионирования.

Рано или поздно вы поддержите несколько версий потребителей одновременно. **Pact Broker** (open-source решение для хранения и версионирования контрактов) или его коммерческая версия **Pactflow** решают эту проблему, храня контракты с тегами версий.

Реальный пример из практики: как сокращают регресс в 4 раза с помощью контрактов

Во многих FinTech- и e-commerce-проектах с **микросервисной архитектурой** команды сталкиваются с одинаковой проблемой: несколько сервисов развиваются параллельно разными командами, а перед релизом интеграционное тестирование превращается в многодневный квест.

Типичная картина: инженеры вручную поднимают общий стенд, прогоняют коллекции Postman, сверяют JSON-структуры, пытаются понять, на чьей стороне «сломался» формат. Такой **регресс** легко занимает два-три дня и при этом не даёт ощущения надёжности — всегда остаётся тревога, что чьи-то изменения незаметно задела соседний сервис.

В подобных ситуациях команды часто внедряют контрактное тестирование с помощью **Pact**.

Подход выглядит так:

- сервис-потребитель в своих тестах формирует контракт (pact-файл) с описанием ожидаемого API;
- этот контракт публикуется в общий репозиторий;
- в CI сервиса-провайдера появляется этап верификации: сборка проверяет, что текущая версия API всё ещё соответствует ожиданиям потребителей;
- если контракт нарушен — сборка падает ещё до попадания изменений на общий стенд.

В результате большая часть проблем интеграции (несовпадение форматов, обязательных полей, кодов ответа) ловится на этапе сборки, а не во время общего регресса.

На практике это приводит к заметному эффекту: существенно сокращается объём регресса, связанного с несовместимостью API.

Интересная практика, встречающаяся в высоконагруженных командах: они дополняют контрактные тесты анализом реального production-трафика. С помощью инструментов вроде **WireMock** записываются реальные запросы и ответы, после чего инженеры находят неочевидные кейсы и добавляют их в тесты потребителей. Так контракты постепенно начинают отражать не только «задуманное» API, но и то, как им реально пользуются в продакшене.

Этот подход позволяет убрать главный страх микросервисных команд перед релизом: «а вдруг у соседей что-то отвалится», заменяя его быстрым и автоматическим сигналом прямо в CI.

Контракты — это не только про JSON

Хотя пример выше с **REST**, контрактное тестирование применимо к любым асинхронным взаимодействиям: сообщения в **Kafka**, RabbitMQ, **gRPC**. Для Kafka, например, Pact поддерживает спецификацию сообщений. Вы описываете, какой формат сообщения ожидает потребитель, и провайдер верифицирует, что продюсит именно такой.

Код контракта для Kafka-сообщения:

```
import au.com.dius.pact.consumer.dsl.PactDslJsonBody
import au.com.dius.pact.consumer.junit5.PactConsumerTestExt
import au.com.dius.pact.consumer.junit5.PactTestFor
import au.com.dius.pact.core.model.annotations.Pact
import au.com.dius.pact.core.model.messaging.MessagePact
import org.junit.jupiter.api.extension.ExtendWith

@ExtendWith(PactConsumerTestExt::class)
```

```

@PactTestFor(providerName = "order-service", providerType = ProviderType.ASYNC)
class OrderMessagePactTest {

    @Pact(consumer = "notification-service")
    fun orderCreatedEvent(builder: MessagePactBuilder): MessagePact {
        return builder
            .given("order created")
            .expectsToReceive("an order created event")
            .withContent(
                PactDslJsonBody()
                    .integerType("orderId", 100)
                    .stringType("status", "CREATED")
            )
            .toPact()
    }
}

```

Объяснить с 

На стороне провайдера проверяется, что реальное сообщение соответствует этому шаблону.

Что дальше? Системный подход к автоматизации на Kotlin

Контрактное тестирование — мощный инструмент, но оно не заменяет ни модульные, ни интеграционные тесты. Это дополнительный слой защиты, который позволяет командам двигаться быстро и независимо.

Когда интеграции начинают ломаться неочевидно, а тесты перестают давать уверенность, возникает простой запрос: проверять поведение системы на уровне реальных взаимодействий, а не отдельных компонентов. Дальше — выстроить полный контур автоматизации: от API и пользовательского интерфейса до нагрузочных сценариев и встраивания проверок в конвейер сборки. Такой подход последовательно разбирается в курсе [«Автоматизатор тестирования на Kotlin»](#) — с фокусом на практику и применимость в реальных проектах.

Пройдите вступительный тест, чтобы узнать, подойдет ли вам программа курса. А чтобы узнать больше о формате обучения и задать свои вопросы, приходите на бесплатные уроки:

- 28 апреля в 20:00. «Контрактные тесты в Kotlin: как подружить фронт и бэкэнд». [Записаться](#)
- 21 мая в 20:00. «Суперсилы Kotlin для удобных UI-автотестов». [Записаться](#)

Теги: [java](#), [ci](#), [микросервисы](#), [kotlin](#), [pact](#), [тестирование api](#), [контрактное тестирование](#)

Хабы: [Блог компании OTUS](#), [Kotlin](#), [Тестирование IT-систем](#), [Микросервисы](#)



Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Электронная почта

Подписаться

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



512K+

Охват за 30 дней

OTUS

Развиваем технологии, обучая их создателей

[Сайт](#) [ВКонтакте](#) [Telegram](#)



64K+

12

Охват за 30 дней

Карма

Sergey Proshchaev [@sproshchaev](#)

Tech Lead | Java | Kotlin developer

Подписаться



Поток Бэкенд доступен 24/7 благодаря поддержке друзей Хабра

Хабр Курсы для бэкендеров

РЕКЛАМА

Практикум, Хекслет, SkyPro, авторские курсы — собрали всех и попросили скидки. Осталось выбрать!

Перейти

[Перейти в поток Бэкенд](#)

[Комментировать](#)

Публикации

[ЛУЧШИЕ ЗА СУТКИ](#)

[ПОХОЖИЕ](#)



the_bat

23 часа назад

Ремонт блока питания с Power Delivery. 470 граммов электроники

Простой

4 мин

16K

+97

31

27



interpres

18 часов назад

Электроинструмент становится хуже, и это делается намеренно

Простой

11 мин

27K

[Обзор](#)

[Перевод](#)

 +73

 46

 87



prohronus

19 часов назад

Как ИИ-агенты стали новым оружием скамеров на Хабр Карьере

 5 мин

 9.5K

Кейс

 +54

 9

 9



alizar

22 часа назад

Готовимся к отключению. Эффективные форматы для упаковки и раздачи HTML-страниц

 Простой

 7 мин

 12K

Обзор

 +36

 61

 18



AlexSerbul

23 часа назад

Программирование с AI-ассистентом — похороните меня под плинтусом

 Средний

 14 мин

 10K

Мнение

 +28

 33

 12



TrexSelectel

19 часов назад

Linux 7.0 и что изменилось в ядре после очередного цикла разработки

 5 мин

 8.9K

Обзор

 +27

 10

 0



mariklolik

19 часов назад

Как переложить нагрузку по code review с разработчиков на LLM

 Средний

 7 мин

 7.6K

Кейс

 +23

 23

 6



nick_dyuba

23 часа назад

Легенды 90-х — кто придумал и производил жвачки Turbo, Love is... и TіріTіr

 5 мин

 7.2K

Recovery Mode

+23

8

5



infgotoinf

11 часов назад

Perl — зря забытый язык программирования?



11 мин



8.8K

Из песочницы

+16

13

10



SLY_G

23 часа назад

Что такое «мышечная память» и можно ли её развить?



4 мин



7.1K

Перевод

+16

10

7

Big Data в фарме: как мы управляем сотнями сотрудников по всей стране

Турбо

Показать еще

МИНУТОЧКУ ВНИМАНИЯ



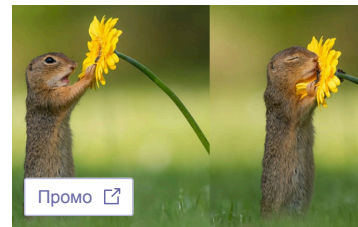
Турбо

Собери облачный пазл и выиграй призы



Турбо

Удалёнка в фарме: большие данные и никакой магии



Промо

Запахло весной и скидками в Промокодусе

ИНФОРМАЦИЯ

Сайт	otus.ru
Дата регистрации	22 марта 2017
Дата основания	1 апреля 2017
Численность	101–200 человек
Местоположение	Россия
Представитель	OTUS

ВИДЖЕТ



В КОНТАКТЕ

 **OTUS: программирование и ...**

29,166 followers

Follow on VK

БЛОГ НА ХАБРЕ

17 часов назад

Автоматизация процессов на open source — n8n и Ollama

 6.8K  4

22 часа назад

Контрактное тестирование на Kotlin: гайд для автоматизатора

 5.1K  0

17 апр в 20:05

Как читать BUFFERS в EXPLAIN ANALYZE и находить I/O-узкие места в PostgreSQL

 6.5K  0

17 апр в 17:40

Как работает движок микрокода процессора 8086

 9.1K  1

17 апр в 13:30

Как ИИ-агенты меняют управление IT-проектами

 6K  3

ИСТОРИИ



Лучшее за неделю
Топ-7 статей из блогов компаний

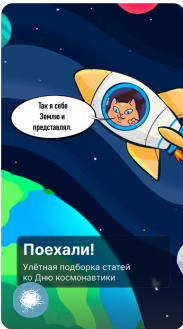


ПРОВЕРЬ СВОЙ IQ
Запускаем инженерный челлендж. Сможешь ли ты решить сложный пазл?



ЧТО ДАЛЬШЕ, НАСТЯ? СЛАБ?

НЕТ, 25 АПРЕЛЯ У НАС КОЕ-ЧТО ПОИНТЕРЕСНЕЕ — АЛФА ЦЕПЬ

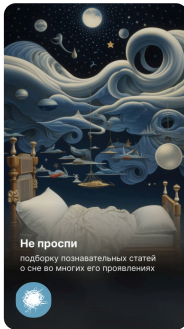


Так я себе Звезда и представляю.

Поехали!
Улётная подборка статей ко Дню космонавтики

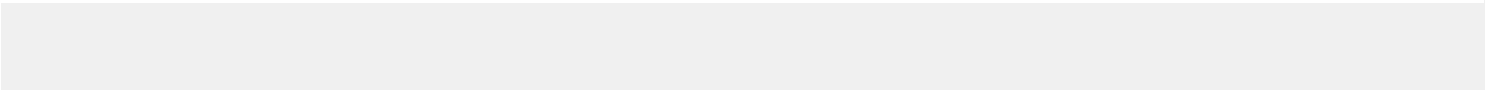
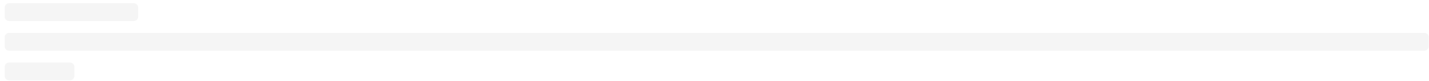
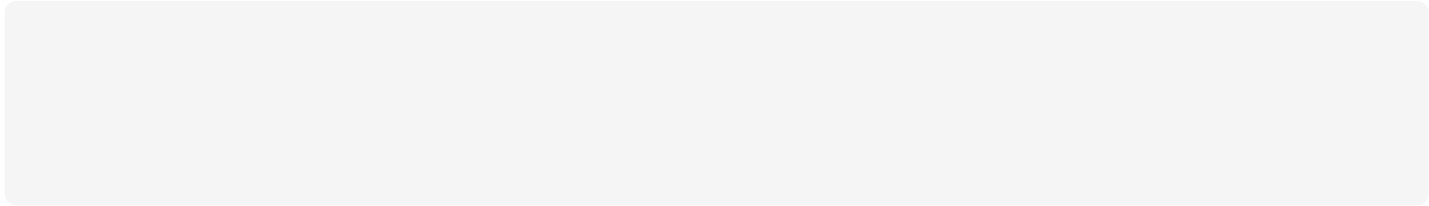


Жизнь животных
Подборка статей об удивительных фактах из мира живой природы



Не просни
подборку познавательных статей о сне во многих его проявлениях





Ваш аккаунт

Войти

Регистрация

Разделы

Статьи

Новости

Хабы

Компании

Авторы

Песочница

Информация

Устройство сайта

Для авторов

Для компаний

Документы

Соглашение

Конфиденциальность

Услуги

Корпоративный блог

Медийная реклама

Нативные проекты

Образовательные программы

Стартапам



Настройка языка

Техническая поддержка

© 2006–2026, Habr

